

Laboratorium: Dynamiczne Formularze i Trwałość Danych we Flask

Aplikacje Internetowe

1 Zadania wprowadzające

Celem laboratorium jest budowa interaktywnego systemu zarządzania danymi. Studenci nauczą się łączyć dynamiczną manipulację drzewem DOM (JavaScript) z przetwarzaniem listowych danych formularzowych po stronie serwera (Flask) oraz zapisem do plików JSON.

Zadanie 1: Struktura danych i odczyt (20 min)

Przygotuj funkcje pomocnicze do obsługi bazy danych w pliku `cv_database.json`.

Wskazówka: Pamiętaj, że w Pythonie wszystkie składowe klasy są publiczne [cite: 8], co ułatwia mapowanie słowników na obiekty.

```
1 import json
2 import os
3
4 DATA_FILE = 'cv_database.json'
5
6 def load_data():
7     if os.path.exists(DATA_FILE):
8         with open(DATA_FILE, 'r', encoding='utf-8') as f:
9             return json.load(f)
10    return []
11
12 def save_data(data):
13     with open(DATA_FILE, 'w', encoding='utf-8') as f:
14         json.dump(data, f, indent=4, ensure_ascii=False)
```

Listing 1: Logika bazy danych w app.py

Zadanie 2: Widok tabeli edytowalnej (30 min)

Stwórz szablon `edytuj_tabele.html`. Wykorzystaj pętlę Jinja2 do wygenerowania wierszy. Kluczowe jest użycie nawiasów kwadratowych w nazwach pól (`name="fullname[]"`), co pozwoli Flaskowi odebrać dane jako listę.

```
1 {% for k in kandydaci %}
2 <tr>
3     <td><input type="text" name="fullname[]" value="{{k.
4     fullname}}"></td>
5     <td><input type="email" name="email[]" value="{{k.
6     email}}"></td>
7     <td><button type="button" onclick="deleteRow(this)">
    Usuń</button></td>
8 </tr>
9 {% endfor %}
```

Listing 2: Fragment tabeli w Jinja2

Zadanie 3: Dynamiczne zarządzanie wierszami (40 min)

Zaimplementuj w JavaScript funkcje pozwalające na interakcję z tabelą bez przeładowania strony.

Wymagania:

1. Funkcja `addRow()` dodająca wiersz na końcu.
2. Funkcja `addAfter(button)` wstawiająca nowy wiersz bezpośrednio pod bieżącym (użyj metody `.after()`).
3. Funkcja `deleteRow(button)` usuwająca wiersz z drzewa DOM.

```
1 function addAfter(button) {
2     const currentRow = button.closest("tr");
3     const newRow = document.createElement("tr");
4     newRow.innerHTML = `
5         <td><input type="text" name="fullname[]"></td>
6         <td><input type="email" name="email[]"></td>
7         <td><button type="button" onclick="deleteRow(this
8     )">Usuń</button></td>
9     `;
10 }
```

```

9     currentRow.after(newRow);
10 }

```

Listing 3: 'Manipulacja DOM w czystym JS'

Zadanie 4: Synchronizacja z serwerem (30 min)

Zaimplementuj trasę /aktualizuj-wszystko, która odbierze listy danych za pomocą `request.form.getlist()`.

Wskazówka: Kolejność elementów w listach zwracanych przez Flask odpowiada kolejności wierszy w tabeli HTML w momencie wysłania formularza.

```

1 @app.route('/aktualizuj-wszystko', methods=['POST'])
2 def aktualizuj_wszystko():
3     names = request.form.getlist('fullname[]')
4     emails = request.form.getlist('email[]')
5
6     # Tworzenie nowej struktury danych
7     nowe_dane = [{"fullname": n, "email": e} for n, e in
8 zip(names, emails) if n.strip()]
9
10    save_data(nowe_dane)
11    return redirect(url_for('index'))

```

Zadania główne

Zadanie 1

Założ nowy projekt aplikacji Python FLASK. W ramach tego projektu będziesz tworzył szkielet aplikacji do obsługi formularzy związanych z rozliczeniem praktyk.

Zadanie 2

Opracuj szkielet formularza do pobierania danych, które wypełniają załącznik 4 "POTWIERDZENIE UZYSKANIA EFEKTÓW UCZENIA SIĘ W RAMACH PRAKTYKI ZAWODOWEJ". Dane zapisuj do JSON. Docelowo dane będą zapisywane do bazy dokumentowej. Każdy tego typu dokument będzie zwieszony dla konkretnego studenta.

Zadanie 3

Opracuj szkielet formularza do pobierania danych, które wypełniają załącznik 6 "DZIENNIK PRAKTYKI ZAWODOWEJ". Podobnie jak w poprzednim przypadku dane są skojarzone z konkretnym studentem. Możesz to uwzględnić w strukturze JSON. Zauważ, że dziennik praktyk ma formę tabeli w której każdy wiersz sprawozdaje pracę wykonaną danego dnia praktyki.